

Utiliser PPCL sur Google Colab

Mode d'emploi = comment exécuter le notebook « Python in the Physical Chemistry Lab » dans le cloud, sans rien installer sur votre PC local

Google Colab permet d'exécuter ce notebook directement dans votre navigateur, sur les serveurs de Google, sans installer Python ni aucune bibliothèque sur votre ordinateur. C'est une alternative pratique à une installation locale de JupyterLab, en particulier sur une machine où vous ne pouvez rien installer. Il vous suffit d'un compte Google.

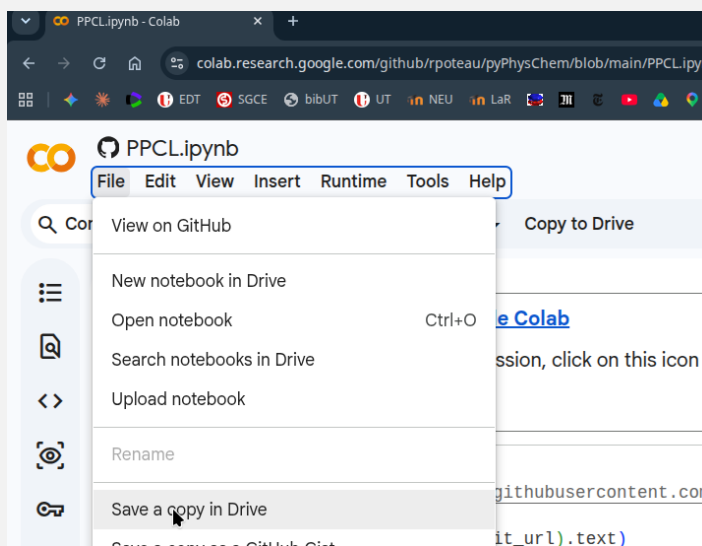
L'objectif de ce notebook. Il ne s'agit **pas** de devenir un pro de l'algorithmique et de Python. L'objectif est de savoir **écrire du code de base**, et surtout de savoir **lire, interpréter, analyser et corriger** un code fourni par ailleurs — par exemple par une **IA générative**. À l'heure où l'on obtient du code en quelques secondes, l'essentiel n'est plus de le produire, mais de **comprendre ce qu'il fait et de juger s'il est correct**.

1. La première fois : ouvrir et s'approprier le notebook

1. Rendez-vous sur la page du notebook : github.com/rpoteau/pyPhysChem/blob/main/PPCL.ipynb
2. Cliquez sur le badge « **Open in Colab** » en haut du notebook. Connectez-vous avec votre compte Google si ce n'est pas déjà fait.



3. Le notebook s'ouvre dans Colab, mais en **lecture seule** (il vient de GitHub). Pour pouvoir sauvegarder votre travail, allez dans File > Save a copy in Drive.

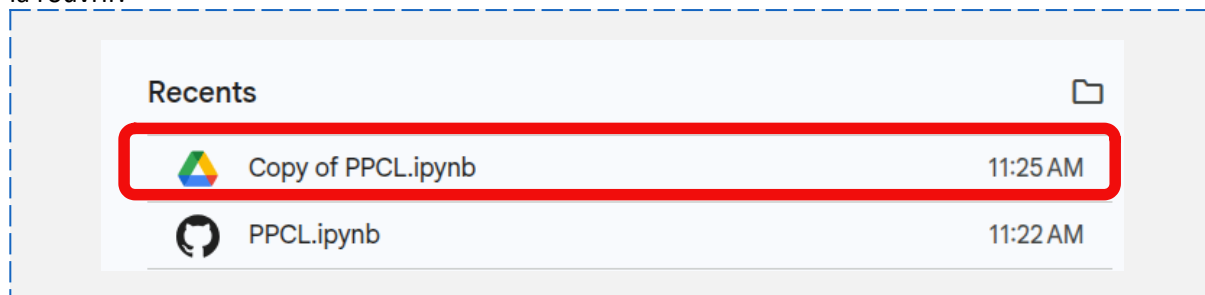


4. Désormais, vous travaillez sur **cette copie**, enregistrée dans votre espace **Google Drive**. C'est elle qu'il faudra rouvrir à chaque séance.

À retenir : cette étape ne se fait qu'**une seule fois**. Ensuite, votre copie personnelle vit dans votre Drive, indépendante de la version GitHub.

2. Reprendre le travail lors d'une séance suivante

1. Allez directement sur colab.research.google.com et connectez-vous.
2. Votre **copie de PPCL** apparaît **en tête des fichiers récents**, sur la page d'accueil. Cliquez dessus pour la rouvrir.



Important : à chaque réouverture, vous devez **ré-exécuter les deux premières cellules d'initialisation** (voir §5). Colab efface l'environnement entre deux sessions.

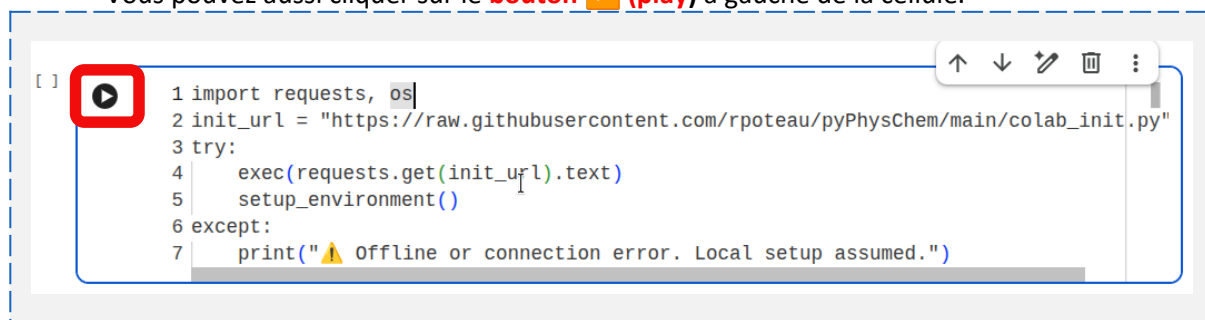
3. Comprendre le document

Ce document est un **notebook** : il mêle dans une même page du **texte**, des **images**, des **équations** et du **code** que vous pouvez exécuter. Il est composé de **cellules**, de deux types :

- **Cellules de code** : pour écrire et exécuter du Python. Le résultat s'affiche juste en dessous.
- **Cellules de texte (Markdown)** : pour les explications, titres, figures et équations. Si vous double-cliquez dessus par erreur, vous verrez le code de mise en forme : ré-exécutez la cellule pour retrouver l'affichage normal.

4. Exécuter les cellules

- Cliquez dans une cellule, puis appuyez sur **Maj + Entrée** (Shift + Enter) : la cellule s'exécute et le curseur passe à la suivante.
- Autre raccourci : **Ctrl + Entrée** exécute la cellule **sans** passer à la suivante.
- Vous pouvez aussi cliquer sur le **bouton ▶ (play)** à gauche de la cellule.



- Pour tout relancer proprement depuis le début : menu Exécution > Tout exécuter (Runtime > Run all).

Bon réflexe : l'ordre d'exécution compte. Un notebook garde en mémoire ce que les cellules **déjà exécutées** ont produit. Si un résultat semble incohérent, réexécutez depuis le haut (ou Exécution > Redémarrer et tout exécuter).

5. Initialiser l'environnement (à faire à chaque session)

Une fois le notebook ouvert, vous devez préparer l'environnement Python et pyPhysChem en exécutant **les deux premières cellules de code, dans l'ordre**. La première **installe les bibliothèques et télécharge les**

fichiers d'accompagnement : elle prend **environ 2 minutes**. Attendez qu'elle soit terminée avant de continuer.

```
[ ] 1 import requests, os
    2 init_url = "https://raw.githubusercontent.com/rpoteau/pyPhysChem/main/colab_init.py"
    3 try:
    4     exec(requests.get(init_url).text)
    5     setup_environment()
    6 except:
    7     print("⚠️ Offline or connection error. Local setup assumed.")

[ ] 1 import pyphyschemtools as t4pPC
    2 from pyphyschemtools import fg, hl, bg, color
    3 t4pPC.init()
    4 %matplotlib inline
    5
    6 __author__ = "Romuald Poteau, LPCNO (INSA-CNRS-UT lab, Toulouse, France)"
```

Pourquoi refaire cela à chaque fois ? Colab tourne sur une machine **temporaire** : après une période d'inactivité ou à la fermeture, elle est réinitialisée. Les bibliothèques installées et les fichiers téléchargés sont alors perdus ; mais votre notebook (code et résultats) reste sauvegardé dans votre Drive.

6. Insérer une nouvelle cellule (texte ou code)

Vous pouvez ajouter vos propres cellules, par exemple pour prendre des notes ou tester du code. Survolez l'espace entre deux cellules (ou le bas d'une cellule) : deux boutons apparaissent, « **+ Code** » et « **+ Texte** ». Cliquez sur celui voulu pour insérer une cellule à cet endroit.

```
1 # Every line that starts with a # character is a comment
2
3 # addition
4 3 + 8 # it is also possible to add a comment after a command
```

+ Code + Text

- **+ Code** insère une cellule de code, exécutable avec Maj + Entrée.
- **+ Texte** insère une cellule de texte (Markdown) : tapez votre texte, puis Maj + Entrée pour l'afficher mis en forme.

7. Applications et exercices

Les applications

Le notebook contient des **applications** : des exemples de code parfois **assez élaborés**. On ne vous demande **pas** de savoir les écrire vous-même. Leur objectif est de vous faire **comprendre ce que fait le code**, d'en saisir la logique et les différentes parties, et surtout de **prendre du recul** pour être capable de **transposer ces idées à d'autres problèmes**.

Les exercices

Le notebook contient aussi des **exercices**, suivis d'une **cellule de code vide** dans laquelle écrire votre réponse. Ceux-ci sont à faire **par vous-même** : autant que possible, **évitons de recourir à une IA générative** — c'est en cherchant que vous acquerez les réflexes de base.

La **solution est systématiquement fournie juste après**, mais masquée. La cellule de solution contient une ligne du type `# %load ./SolutionsToExercices/...` Pour la révéler :

- enlevez le commentaire de la ligne %load (raccourci clavier utile : Ctrl + /) ;
- **exécutez la cellule une première fois** (Maj + Entrée) : cela charge le corrigé dans la cellule ;
- **exécutez-la une seconde fois** : cela fait tourner le code et affiche le résultat.

Le bon usage : essayez d'abord **sérieusement**, puis comparez avec la solution. Révéler la réponse trop tôt vous prive de l'essentiel — apprendre à écrire et à corriger le code vous-même.

8. Sauvegarder votre travail

Une fois votre copie créée dans le Drive (§1), enregistrez à tout moment avec **Ctrl + S** ou File > Save. Votre code et les résultats affichés sont conservés. En revanche, le **contenu de la mémoire** (les variables) ne se sauvegarde pas : à la réouverture, réexécutez les cellules depuis le début.

Prêt ? Posez votre téléphone 📵 et entrons dans le monde de Python. 🐍🔥